3606 assignment

Github link: https://github.com/notnth/dave3606-project-2026/tree/main

Gruppemedlemmer:

- Nhu Thien Huynh, nhhuy4829@oslomet.no, github: notnth
- Julie Marie Skaarer Kreutz, jukre5057@oslomet.no, github: juliemariesk

Task 1

The first primary key added for id in lego_set. This key was added because each lego set has a unique ID, thus speeding the queries.

```
lego-db=# ALTER TABLE lego_set
lego-db-# ADD PRIMARY KEY(id);
ALTER TABLE
```

Next primary key added was for (brick_type_id, color_id) in lego_brick. This primary key was added to ensure that every brick had its own unique key (due to brick type and color).

```
lego-db=# ALTER TABLE lego_brick
lego-db-# ADD PRIMARY KEY(brick_type_id, color_id)
ALTER TABLE
```

Last primary key added was for (set_id, brick_type_id, color_id) in lego_inventory. This primary key was added to ensure that each row will represent a specific brick (brick type and color). Every brick is unique.

```
lego-db=# ALTER TABLE lego_inventory
lego-db-# ADD PRIMARY KEY(set_id, brick_type_id, color_id);
ALTER TABLE
```

For foreign key we added a link between lego_inventory and lego_set(id). This was done to ensure that it is impossible to add a new row that doesn't exist.

```
lego-db=# ALTER TABLE lego_inventory
lego-db-# ADD CONSTRAINT fk_inventory_set
lego-db-# FOREIGN KEY(set_id)
lego-db-# REFERENCES lego_set(id);
ALTER TABLE
```

Next foreign key ensures that each inventory entry refers to an existing brick which makes it impossible to add a new tow that doesn't exist.

-
```
lego-db=# ALTER TABLE lego_inventory
lego-db-# ADD CONSTRAINT fk_inventory_brick
lego-db-# FOREIGN KEY(brick_type_id, color_id)
lego-db-# REFERENCES lego_brick(brick_type_id, color_id);
ALTER TABLE
```

Here's an overview of all the keys:

-
```
lego-db=# \d lego_set
                    Table "public.lego_set"
      Column       |  Type   | Collation | Nullable | Default
-------------------+---------+-----------+----------+---------
 id                | text    |           | not null |
 name              | text    |           | not null |
 year              | integer |           |          |
 category          | text    |           |          |
 preview_image_url | text    |           |          |
Indexes:
    "lego_set_pkey" PRIMARY KEY, btree (id)
Referenced by:
    TABLE "lego_inventory" CONSTRAINT "fk_inventory_set" FOREIGN KEY (se
t_id) REFERENCES lego_set(id)
```

Task 2

- Which LEGO sets contain a specific brick type, regardless of color?
  - SELECT DISTINCT set_id FROM lego_inventory WHERE brick_type_id = ‘3001’;
- Which LEGO sets contain bricks of a specific color, regardless of type
  - SELECT DISTINCT set_id FROM lego_inventory WHERE color_id = 5;
- Next task: Create index to improve time for queries. These are the statements needed to create the index:
  - CREATE INDEX idx_inventory_brick_type_set ON lego_inventory(brick_type_id, set_id);
  - CREATE INDEX idx_inventory_color_set ON lego_inventory(color_id, set_id);

Why were the queries faster after using indexes?
Answer:

-
```
lego-db=# SELECT DISTINCT set_id
FROM lego_inventory
WHERE brick_type_id = '3001';
Time: 163.815 ms
lego-db=# CREATE INDEX idx_inventory_brick_type_set
ON lego_inventory (brick_type_id, set_id);
CREATE INDEX
Time: 1309.317 ms (00:01.309)
lego-db=# SELECT DISTINCT set_id
FROM lego_inventory
WHERE brick_type_id = '3001';
Time: 3.132 ms
lego-db=# SELECT DISTINCT set_id FROM lego_inventory WHERE color_id = 5;
Time: 239.664 ms
lego-db=# CREATE INDEX idx_inventory_color_set
ON lego_inventory (color_id, set_id);
CREATE INDEX
Time: 758.324 ms
lego-db=# SELECT DISTINCT set_id FROM lego_inventory WHERE color_id = 5;
Time: 11.792 ms
```

The results were as followed:

- Before query for brick_type: 163.815 ms
- After query for brick_type: 3.132 ms
- Before query for color_id: 239.664 ms
- After query for color_id: 11.792 ms

To first do this task, I made two queries: one for brick type and one for color. These queries would find unique (distinct) set_ids of bricks regardless of type or color within the lego_inveotry table. Next, I followed the task which asked to create indexes to make the queries more efficient. For this task, we needed to compare time meaning I used \timing to be able to see the retrieval time for each query. The goal here is to see the query times before creating the index and after. To create the index for each query, I created one index for (brick_type_id, set_id) and one for (color_id, set_id). This means that PostgreSQL can perform an index-only scan, a way where data is retrieved directly from an index without the need to access the table. After this, I used the same queries again and compared the time difference. As seen from the results above, the time increased by a ton, meaning the amount of data that needed to be read and processed reduced, thus leading to a faster query execution.

Task 3

Here we were asked to improve the performance of the /sets endpoint. This endpoint retrieves all LEGO sets from the database and generates HTML to display them.

The original solution built to the HTML using string concatenation inside a loop is inefficient in Python because strings are immutable. Every time a row is added a new string is created in memory. When there are many rowsthis will lead to a lot of unnecessary copying and slows down the program. To fix this i changed the implementation to use a list and join() :

Row_parts []

For row in cur.fetchall():

Row_parts.append(f":...")

Rows = "".join(row_parts)

This solution is more efficient because each row is added to a list which is a fast operation. Then all the rows are combined into one string at the end using join(): this avoids creating new strings repeatedly.

To test the performance i used perf_counter() to measure the time it took to render all sets. The results showed that the endpoint worked efficiently after the change.

The improved solution has a time complexity of O(n) while the original solution was closer to O(n^2) because og repeated string copying.

To verify the solution i started the server and opened the /sets endpoint in the browser. The page correctly displayed all LEGO sets as shown here:

All Lego sets

Id	Name
00-1	Weetabix Castle
001-1	Gears
0011-2	LEGOLAND Mini-Figures
0011-3	Castle 2 for 1 Bonus Offer
0012-1	Red Spacemen
0013-1	White Spacemen
0014-1	Yellow Spacemen
0015-1	Assorted Spacemen
0016-1	Knights
00-2	Weetabix Promotional House 1
002-1	Motor Pak (4.5V Samsonite Gears Motor Set)
00-3	Weetabix Promotional House 2
003-1	Master Mechanic Set
00-4	Weetabix Promotional Windmill
004-1	Master Builder Set
00-5	Puppet Set - Quaker/Life Cereal
005-1	Basic Building Set in Cardboard

I also measured the rendering time using perf_counter(). The terminal output showed:

```
 * Debugger PIN: 694-138-088
Time to render all sets: 0.1686356250000074
127.0.0.1 - - [26/Mar/2026 14:55:47] "GET /sets HT
```

This confirms that the endpoint works efficiently after the optimalization.

Task 4

Here we modified the /sets endpoints to support UTF-8 and UTF-16. This lets it accept query parameters called encoding. Example:

- /sets?encoding=utf-8
- /sets?encoding=utf-16

First, import gzip

- import gzip

The encoding paremeter is read via:

- encoding = request.args.get("encoding", "utf-8")

- if encoding not in ["utf-8", "utf-16"]:
- encoding = "utf-8"

We must also change the meta charset when it’s not UTF-8. Here we changed it from

- <meta charset="UTF-8"> to {META_CHARSET}

We also add this on whether the tag should be included:

- meta_charset = '<meta charset="UTF-8">' if encoding == "utf-8" else ""

Then the template with:

- return template.replace("{META_CHARSET}", meta_charset).replace("{ROWS}", rows)

Next, we need to fix /set endpoint. The HTML string must be encoded before it’s sent to the browser:

- html_output = get_all_sets_html(DB, meta_charset)
-
- body = html_output.encode(encoding)

- return Response(
- body,
- content_type=f"text/html; charset={encoding}"
-)

After it’s encoded, the byte array is compressed:

- body = html_output.encode(encoding)

- compressed_body = gzip.compress(body)

The response is then sent to the right header:

- return Response(
- compressed_body,
- content_type=f"text/html; charset={encoding}",
- headers={"Content-Encoding": "gzip"},
-)

The browser should do the rest of the work and decompress the response.

Task 5

First, we verify the existing /api/ set endpoint so that it returns a complete lego set (and not just id). The JSON contains set id, name, year, category and an inventory list with brick_type_id, color_id and count.

```
- @app.route("/api/set")
- def api_set():
-   set_id = request.args.get("id")
-   json_output = get_set_json(DB, set_id)
-   return Response(json_output, content_type="application/json")
```

The JSON data is made by the get_set_json function, which queries the database to get both metadata and inventory. It constructs a dictionary based on this into JSON formatting.

```
- def get_set_json(database, set_id):
-   query_set = "SELECT id, name, year, category FROM lego_set WHERE id = %s"
-   query_inventory = "SELECT brick_type_id, color_id, count FROM lego_inventory
  WHERE set_id = %s"

-   set_rows = database.execute_and_fetch_all(query_set, (set_id,))
-   if not set_rows:
-     return json.dumps({"error": "Set not found"}, indent=4)



-   set_data = {
-     "set": {
-       "id": set_rows[0][0],
-       "name": set_rows[0][1],
-       "year": set_rows[0][2],
-       "category": set_rows[0][3],
-     },
-     "inventory": [],
-   }

-   for brick in database.execute_and_fetch_all(query_inventory, (set_id,)):
-     set_data["inventory"].append({
-       "brick_type_id": brick[0],
-       "color_id": brick[1],
-       "count": brick[2],
-     })
```

```
-     return json.dumps(set_data, indent=4)
```

After that, we implemented a new endpoint in /api/setfile which exports the same data to a byte-based format.

```
-   @app.route("/api/setfile")
-   def api_setfile():
-     set_id = request.args.get("id")
-     binary_output = get_set_binary(DB, set_id)
-     return Response(binary_output, content_type="application/octet-stream")
```

Here is a custom file format encoded in UTF-8:

```
-   SET;set_id;name;year;category
-   BRICK;brick_type_id;color_id;count
```

The binary format builds the lines of the file, joins them into one string and converts the result into bytes:

```
-   def get_set_binary(database, set_id):
-     query_set = "SELECT id, name, year, category FROM lego_set WHERE id = %s"
-     query_inventory = "SELECT brick_type_id, color_id, count FROM lego_inventory
    WHERE set_id = %s"

-     set_rows = database.execute_and_fetch_all(query_set, (set_id,))
-     if not set_rows:
-       return b"ERROR: Set not found"

-     set_id, name, year, category = set_rows[0]

-     lines = []
-     lines.append(f"SET;{set_id};{name};{year};{category}")

-     inventory_rows = database.execute_and_fetch_all(query_inventory, (set_id,))
-     for brick_type_id, color_id, count in inventory_rows:
-       lines.append(f"BRICK;{brick_type_id};{color_id};{count}")

-     text = "\n".join(lines)
-     return text.encode("utf-8")
```

Lastly, we implemented a small console that can the file format and prints the lego set info:

```
import sys

def main():
    if len(sys.argv) < 2:
        print("Usage: python3 read_set_file.py <filename>")
        return

    filename = sys.argv[1]

    with open(filename, "rb") as f:
        data = f.read()

    text = data.decode("utf-8")
    lines = text.splitlines()

    for line in lines:
        parts = line.split(";")

        if parts[0] == "SET":
            set_id = parts[1]
            name = parts[2]
            year = parts[3]
            category = parts[4]

            print(f"Set ID: {set_id}")
            print(f"Name: {name}")
            print(f"Year: {year}")
            print(f"Category: {category}")
            print()
            print("Inventory")

        elif parts[0] == "BRICK":
            brick_type_id = parts[1]
            color_id = parts[2]
            count = parts[3]

            print(f"Brick type: {brick_type_id}, Color: {color_id}, Count: {count}")

if __name__ == "__main__":
    main()
```

Task 6

In this task i implemented both frontend functionality and caching to improve performance and usability of theapplication.

The frontend was implemented in set.html using JavaScript. The page retrieves data from the /api/set endpoint by usingthe fetch function and passing the set id as a query parameter. The endpoint returns JSON data containing both the setinformatrion and its inventory. The JSON structure consists of a “set” object and an “inventory” list. The JavaScript codethen dynamically updates the HTML page by inserting the set id and set name into the document. It also iterates throughthe inventory list and creates a new table row for each brick.

This approach allows the page to be dynamically populated without reloading the entire page and it separates thefrontend from the backend logic.

To improve performance i implemented a server-side cache using Pythons OrderedDict. The cache stores the 100 most recently requested Lego sets, including both set metadata and inventory. When a request is made to /api/set the system first checks if the set id exists in the cache, if the set is found the cached result is returned immediately, if not the data is retrieved from the database and then added to the cache.

The cache uses a Least Recently Used eviction policy. This means that when the cache exceeds 100 entries the least recently used item is removed. This is efficiently handled by OrderedDict, where items are moved to the end when accessed and the oldest item can be removed in constant time.

The time complexity for both accessing and evicting items in the cache is O(1) making this an efficient caching solution.

In addition to server-side cachin i added a Cache-Control header to the /sets endpoint: Cache-Control: public, max-age=60.

This allows the browser to cache the page for up to 60 seconds. As a result repeated requests within this time period do not require a new request to the server, significantly improving performance and reducing server load.

With caching enabled the cached requests to /api/set are significantly faster since they avoid database queries and the /sets page loads faster on repeated visits due to browser caching. Overall caching improves both response time and scalability of the application.

Task 7

In this task i focused on improving the structure of the code and making it easier to test. In the original version the endpoint handlers in server.py were directly connected to the database and SQL queries were written inside route functions. This made the code hard to test and not very flexible.

To fix this i refactored the endpoints by miving the logic into separate functions that take a database instance as a parameter. This means the functions are no longer dependent on a specific database connection and can instead work with different types of databases, including a mock database. It also made the code cleaner and easier to understand since eachfunction now has a clear responsibility. For example get_all_sets_html handles HTML for all sets while get_set_html and get_set_json handle a single set.

Another issue was that psycopg.connect was used directly in each endpoint. This is not ideal since it creates a new connection every time and makes testing more difficult. To improve this i created a Database class in database.py that handles connecting to the database, running queries, fetching results and closing the connection. This way the endpoints dont have to deal with these details and it also becomes easier to swapin a mock database when testing.

For testing i wanted to avoid using a real database. Therefore i created a MockDatabase class in test_server.py that returns predefined data based on the query. This makes it possible to test the logic without relying on an actual database connection.

I then wrote tests using Pythons unittest module to check that the HTML and JSON output was correct. I created four tests, one for all sets, one for a single set in HTML, one for JSON output and one for handling the case where a set is noe found. All the tests passed which shows that the changes work as expected.